CSE 318 — Online 4

Section C2

Your current greedy always picks the single nearest unvisited city. That is fully deterministic, so it can get stuck in one bad tour with no way to escape. In this task you will add a small amount of randomness to the greedy choice and run the construction several times, keeping the best tour.

What to implement.

1. Write a new function `semiGreedyTSP(k)` that behaves like the original `greedyTSP()` except:
   - At each step from the current city, sort the unvisited cities by cost and keep the k **nearest ones** (*Restricted Candidate List*, or RCL). If fewer than k unvisited cities remain, use all of them.
   - Pick one city from the RCL **uniformly at random** as the next city.
2. Write a driver `multiStartSemiGreedy(k, R)` that runs `semiGreedyTSP(k)` **R times** (with a different random seed each time) and returns the *best* tour among the R runs.
3. Use default parameters $k = 7$ and $R = 13$. (yes, you may be lucky and unlucky at the same time!)
4. The tour must still start and end at city 0 (that part is unchanged).

What to show in the output.

- The **best, average, and worst** tour cost across the R runs.
- The best tour itself, printed as `0 -> ... -> 0`.
- A comparison line: original deterministic Nearest Neighbor cost vs. your best semi-greedy cost on the same instance.
- Run the experiment on **at least two** of your test instances.